## Section 1: Signed Numbers and Patterns - /30 Points

1. What is the maximum number of patterns able to be stored in a 5-bit number? How many of those patterns represent negative numbers in 2's Complement?
2. Convert the following to **4-bit Signed Magnitude**:
    - a) -5
    - b) -7
    - c) +6
3. Convert the following to **4-bit 1's Complement**:
    - a) -3
    - b) -9
    - c) -1
4. Convert the following to **4-bit 2's Complement**:
    - a) -4
    - b) -7
    - c) -1
5. Convert the following 4-bit 2's Complement numbers back to base-10:
    - a) `1101`
    - b) `1000`
    - c) `1011`
6. Perform **4-bit 2's Complement addition** on the following. Identify whether overflow occurs:
    - a) $-3+5$
    - b) $-6+-4$
    - c) $7+2$
7. Perform **4-bit 2's Complement subtraction** on the following. Remember: $a-b=a+-(b)$
    - a) $5-3$
    - b) $2-6$
    - c) $-4-3$
8. You are given an encoding scheme that maps 3-bit patterns to the following values:

| Binary | Encoded Value |
| --- | --- |
| 000 | A |
| 001 | B |
| 010 | C |
| 011 | D |
| 100 | E |
| 101 | F |
| 110 | G |
| 111 | H |

- a) How many patterns are available?
- b) If you increment the binary `111` by 1, what happens? What is this called? If we read `111` as the encoding and then added 1, what transition of the symbols would occur?
- c) If you needed to encode 10 distinct values, what is the minimum number of bits required?

---

## Section 2: Boolean Logic and Gates - /20 points

1. Write the boolean expression for each of the following gate combinations (simplify if you can):
   - a) NOT(A AND B)
   - b) (A OR B) AND (NOT A)
   - c) NOT(NOT A AND NOT B)
2. Convert the following boolean expressions to circuit diagrams (draw the gates and label all wires):
   - a) `Out = (A AND B) OR (NOT A AND C)`
   - b) `Out = NOT(A OR B) AND C`

- c) `Out = A XOR (B AND C)`

3. Explain in 1-2 sentences what **functional completeness** means, and give an example of a functionally complete set of gates.
4. Show how to build an **AND gate** using only NAND gates. Provide the circuit diagram and verify with a truth table.
5. Show how to build an **OR gate** using only NAND gates.

---

## Section 3: Truth Tables to Circuits - /20 points

1. Implement the following truth table into a circuit. Derive the boolean expression first, then draw the circuit:

| A | B | C | Out |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

1. Implement the following two-output truth table into a circuit. Derive boolean expressions for **each output separately**:

| A | B | C | Out1 | Out2 |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 1 |

- Hint: Look at what patterns each output follows relative to the inputs. Do any familiar operations come to mind?

---

## Section 4: Basic Circuits - /20 points

1. Draw the truth table for a **Half Adder**. Label the outputs `Sum` and `Carry`. What basic gates make up a half adder?
2. Explain in your own words why a **Full Adder** is needed over a Half Adder. What extra input does it have and what is it used for?
3. Given a **Ripple Carry Adder** built from full adders:
   - a) How many full adders would you need to add two 8-bit numbers?
   - b) Why is it called a "ripple carry" adder?
   - c) What happens if the final carry-out is 1 when working in 2's Complement?
   - Fill in the output column of the following **MUX truth table** (2-to-1 MUX with select line S, inputs A and B):

| S | A | B | Out |
|---|---|---|---|
| 0 | 0 | 0 | |
| 0 | 0 | 1 | |
| 0 | 1 | 0 | |
| 0 | 1 | 1 | |
| 1 | 0 | 0 | |
| 1 | 0 | 1 | |
| 1 | 1 | 0 | |
| 1 | 1 | 1 | |

Write the boolean expression for a 2-to-1 MUX.

1. A **2-to-4 Decoder** takes a 2-bit input and activates exactly one of 4 output lines. Fill in its truth table and write the boolean expression for each output line:

| A | B | Out0 | Out1 | Out2 | Out3 |
|---|---|---|---|---|---|
| 0 | 0 | | | | |
| 0 | 1 | | | | |
| 1 | 0 | | | | |
| 1 | 1 | | | | |

---

## Section 5: Design Problems - /25 points

1. Design a **2-bit ALU** with 3 operations controlled by a 2-bit Op signal. Inputs are 2-bit numbers A and B:
   - Op = `00` : Output A AND B (bitwise)
   - Op = `01` : Output A OR B (bitwise)
   - Op = `10` : Output A + B (addition, ignore carry-out)

   You may use circuit symbols for Half/Full Adders and MUXes. Label all input/output lines and control signals.
2. Design a circuit that takes a 2-bit binary number as input and outputs its **2's Complement negation** as a 2-bit number (ignore overflow). Show your work:
   - a) Write out the truth table for all input combinations
   - b) Derive the boolean expression for each output bit
   - c) Draw the circuit
   - A **4-bit Ripple Carry Adder** is used to add two 4-bit 2's Complement numbers.
   - a) What is the range of numbers each input can represent?
   - b) Add `0111 + 0001` . Does overflow occur? How can you detect overflow in a 2's Complement adder?
   - c) Add `1010 + 1101` . What is the result in base-10?

## Point Summary

| Section | Points |
|---|---|
| Section 1: Signed Numbers and Patterns | /30 |
| Section 2: Boolean Logic and Gates | /20 |
| Section 3: Truth Tables to Circuits | /20 |
| Section 4: Basic Circuits | /20 |
| Section 5: Design Problems | /25 |
| **Total** | **/115** |